



Experiment 06

Conditional Aggregation & Hierarchical Tree Classification in SQL

Name: Deven

UID: 24BCF10060

Aim

To study and implement advanced SQL techniques such as conditional aggregation for comparative daily reports, and self-referential queries for classifying nodes in a hierarchical (tree) structure.

Question 1 – Apples & Oranges Daily Sales Difference (Medium)

A fruit distribution company records the number of apples and oranges sold each day. For every date, exactly one record exists for apples and one record exists for oranges.

Generate a daily sales comparison report showing the difference between the number of apples sold and the number of oranges sold on the same day:

$$\text{Difference} = \text{Apples Sold} - \text{Oranges Sold}$$

Display the sale date and the calculated difference. The result should be sorted by sale date in ascending order.

Given Data

```
1 CREATE TABLE Sales
2 (
3     sale_date DATE,
4     fruit     VARCHAR(10),
5     sold_num  INT
6 );
7
8 INSERT INTO Sales VALUES
9 ('2020-05-01', 'apples', 10),
10 ('2020-05-01', 'oranges', 8),
11 ('2020-05-02', 'apples', 15),
12 ('2020-05-02', 'oranges', 15),
13 ('2020-05-03', 'apples', 20),
14 ('2020-05-03', 'oranges', 0),
15 ('2020-05-04', 'apples', 15),
16 ('2020-05-04', 'oranges', 16);
```

Answer

```
1 SELECT
2     sale_date,
3
4     SUM(CASE WHEN fruit = 'apples' THEN sold_num ELSE 0 END)
5     - SUM(CASE WHEN fruit = 'oranges' THEN sold_num ELSE 0 END) AS diff
6
7 FROM Sales
```



```
8  
9 GROUP BY sale_date  
10  
11 ORDER BY sale_date;
```

Output

sale_date	diff
2020-05-01	2
2020-05-02	0
2020-05-03	20
2020-05-04	-1



Question 2 – Tree Node Classification (Hard)

A company stores hierarchical employee reporting information in a table. Each record represents a node in a hierarchy where `id` represents the current node and `p_id` represents the parent node. Classify every node into one of the following categories:

- **Root** – a node that does not have any parent.
- **Leaf** – a node that has a parent but does not have any children.
- **Inner** – a node that has both a parent and at least one child.

Display the node ID along with its category. Return the result sorted by node ID.

Given Data

```
1 CREATE TABLE Tree
2 (
3     id     INT PRIMARY KEY,
4     p_id  INT
5 );
6
7 INSERT INTO Tree VALUES
8 (1, NULL),
9 (2, 1),
10 (3, 1),
11 (4, 2),
12 (5, 2);
```

Answer

```
1 SELECT
2 id,
3
4 CASE
5     WHEN p_id IS NULL THEN 'Root'
6     WHEN id IN (SELECT DISTINCT p_id FROM Tree WHERE p_id IS NOT NULL)
7         THEN 'Inner'
8     ELSE 'Leaf'
9 END AS type
10 FROM Tree
11
12 ORDER BY id;
```

Output

id	type
1	Root
2	Inner
3	Leaf
4	Leaf
5	Leaf



Learning Outcome

After completing this experiment, the following learning outcomes were achieved.

- Understood how to build comparative daily reports using conditional aggregation with **CASE** inside **SUM()**.
- Learned to pivot row-wise category data (apples/oranges) into column-wise comparisons using **GROUP BY**.
- Applied self-referential subqueries to determine parent-child relationships within a single table.
- Implemented multi-branch **CASE** logic to classify records into Root, Leaf, and Inner categories.
- Strengthened understanding of hierarchical (tree/graph) data modelling and traversal logic in SQL.
- Gained practical exposure to solving real-world reporting and organisational-hierarchy problems using PostgreSQL.